

# Contract Bridge

## ENSC 251 Project Report

Members: Jayden Chan, Matthew Bennett, Arianna Suen

# Table of Contents:

Section A: Final Architecture (pg. 3-8)

Section B: Design Changes (pg. 9)

Section C: Testing and Validation (pg.9)

Section D: Lessons learned (pg. 9)

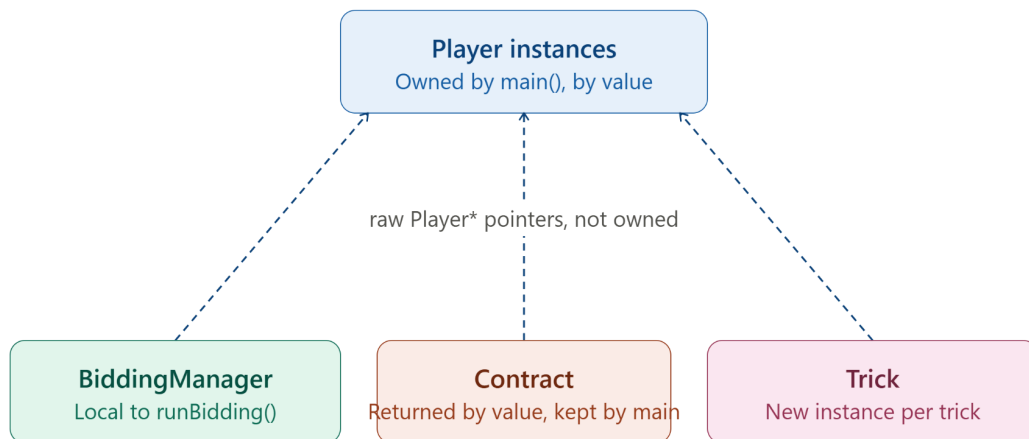
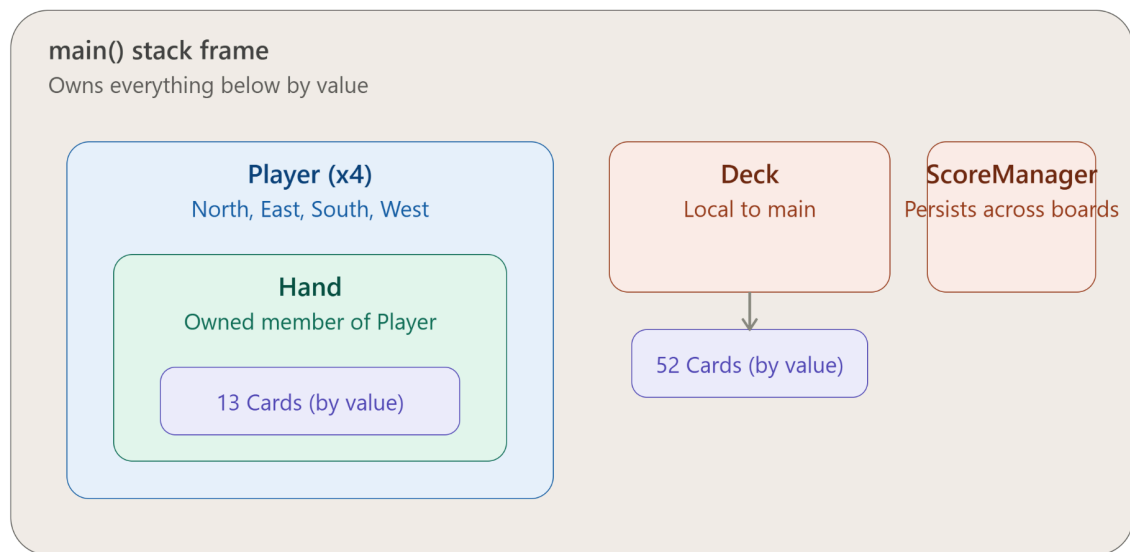
Section E: Future Improvements (pg. 10)

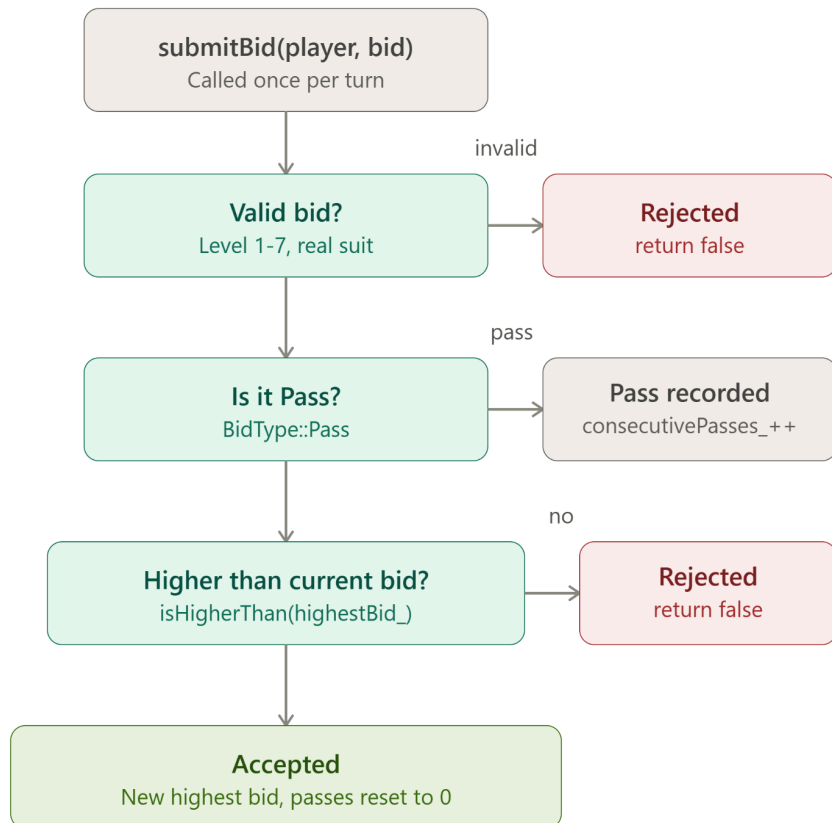
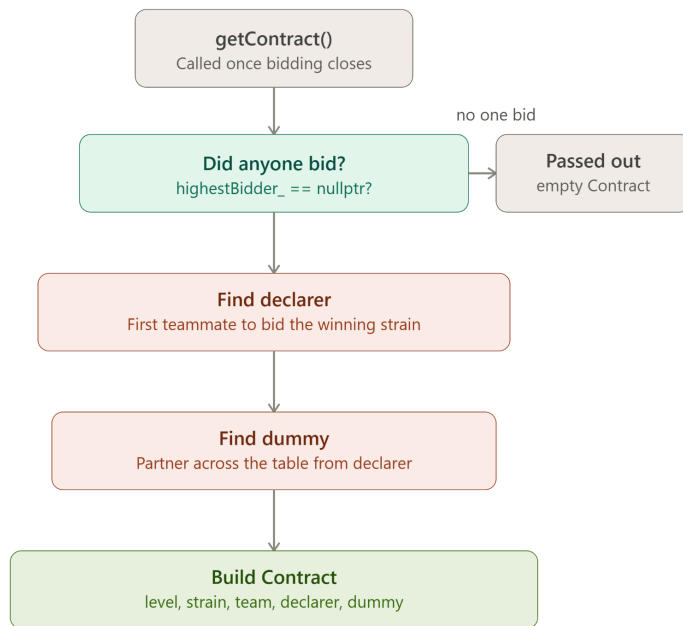
Section F: Game Rules (pg. 11)

Section G: Team Responsibilities (pg. 12)

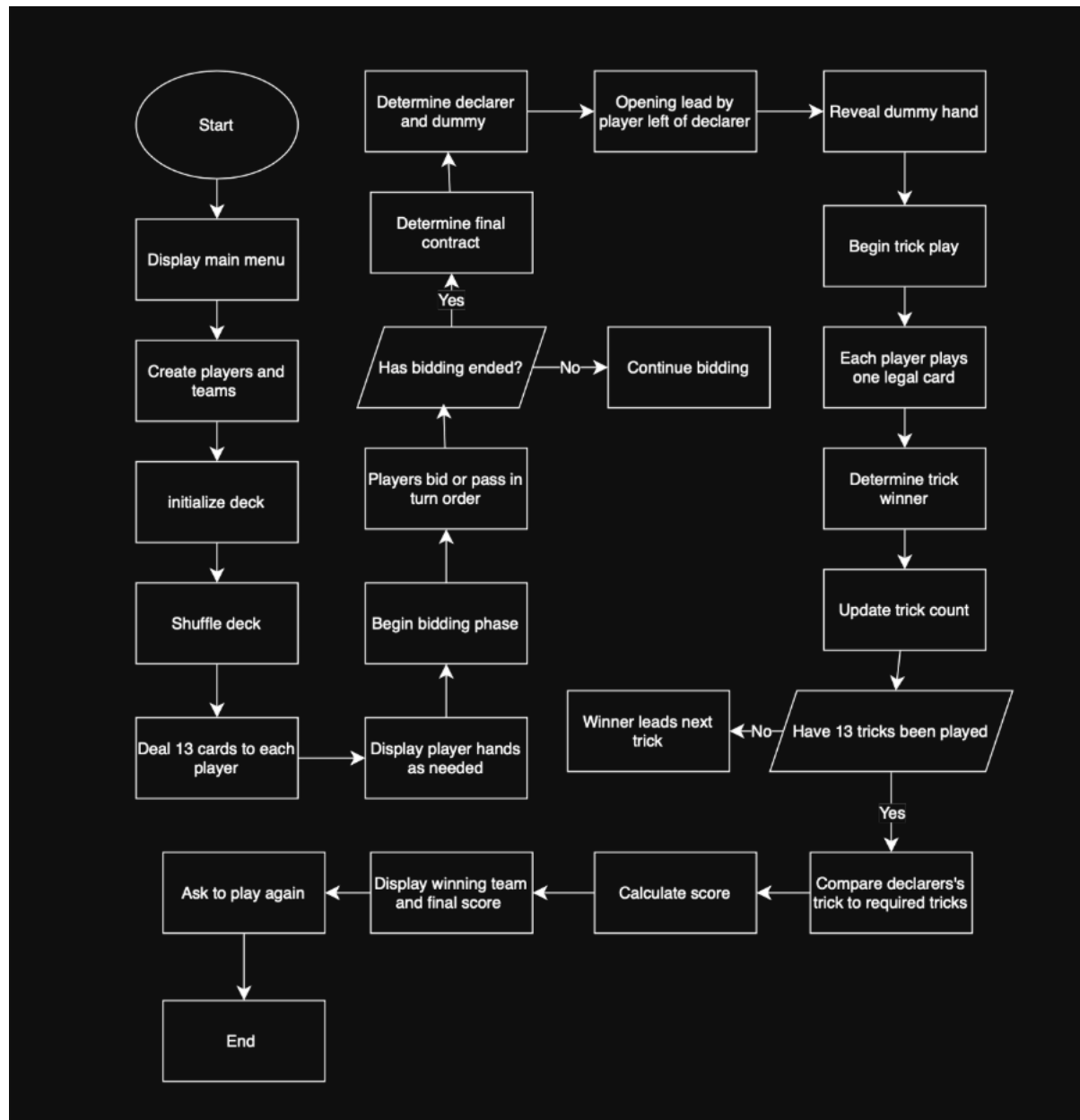
Section H: Appendix (pg. 13-23)

## Section A: Final Architecture





# Main Project Flow



# Classes:

For detailed description of classes, please see appendix.

## 1. Common Class (GR I, II, III)

Contains enums shared by all classes. We needed a way to organize rank and suits, and using enumerations allows us to compare meaningful labels instead of arbitrary integer values, which can remain implicit as the programming logic does not depend on them. We decided not to use inheritance, because a card has ranks and suits but ranks and suits are not cards. As a result, the code becomes more readable and less error prone when comparing specific cards.

Does not contain any objects.

## 2. Card (GR I)

Object: *card*. Represents the smallest unit in the game.

## 3. Deck (GR I)

Object: *cards*. Is a standard deck of 52 playing cards to shuffle, draw, and deal cards. Is a dynamic collection of card objects. (destructor is necessary because hand is dynamically allocated memory )

## 4. Hand (GR III, VI, VIII)

Object: *hand*. Represents the cards currently held by one player. Stores a dynamic collection of *Card* objects.

## 5. Player (GR II, III, VII, VIII)

Object: *player*. Represents one of the four players in the Bridge game. Each player has a name, position, team, and a *Hand* containing their cards.

## 6. Bid (GR IV, V)

Object: *bid*. Represents a player's bid during the bidding phase of Contract Bridge. Stores the **bid level, suit/strain, and bid type** (normal bid or pass).

## 7. Biding Manager (GR II, IV, V, VI)

Object: *bidding manager*. Manages the **bidding phase** of the Bridge game. Keeps track of the players, current highest bid, player who made the highest bid, consecutive passes, and complete bid history.

## 8. Contract (GR VI)

Object: *contract*. Represents the final contract established after the bidding phase. Stores the **contract level, trump suit, declaring team, declarer, dummy, and number of tricks required**.

## 9. Trick (GR VI, VIII, VIII, IX, X)

Object: *trick*. Represents one round of four cards played by the four players. Keeps track of the **cards played, players who played them, lead suit, and trump suit**. Determines which player wins the trick.

## 9. Main GR I-XII)

Object: *game controller*. Coordinates the **Deck, Player, Hand, Bid, BiddingManager, Contract, Trick**, and **ScoreManager** classes to run a complete game of Contract Bridge.

## Section B: Design Changes

Minimal design changes, besides removing Game class. It has been shifted to the main file. No planned improvements were implemented.

## Section C: Testing and Validation

The testing for this project was tedious. Each member function needed to be tested, and due to the interconnectivity of the classes, it was extremely difficult to pinpoint where each error was, especially logical errors.

The trivial tests were that of the setters, getters, display functions, and card comparisons. The most important tests heavily relied on the BiddingManger class. If the contract logic did not work, then the entire game would not work. While the final code uses a randomly generated deck each time, during the testing phases, we set one randomly generated deck as the fixed deck and decreased the number of rounds.

## Section D: Lessons Learned

During the Group project, everyone made their best effort to contribute and write code. The project adheres fully to the minimal viable game agreed upon in the proposal and is playable.

Some conceptual difficulties we encountered were differences in naming convention and interpretation of the rules of the game. Contract bridge itself is a complex game by nature, and it was difficult to determine which rules were absolutely necessary. We omitted the doubling/redoubling and honors-bonus rules, and focused on implementing the core trick-taking and standard scoring. If we were to complete the project again, we would spend more time playing the game for ourselves in the beginning and write the logic as we played, which would have saved time during the planning of the class architecture and member functions.

It was also difficult for us to work with nine classes and cross reference which classes owned the other classes and which classes were only interacting. For example, the game Main owns both the Players and the Deck separately. The Player then owns a Hand, and Hand owns Cards. Deck owns its own separate set of Cards from the pre-shuffle pool. So having different instances of the same class was also difficult to navigate.

For additional lessons learned on the technical side, please see difficulties outlined in the Testing and Validation section above.

# Section E: Future Improvements

We could have implemented the original list of planned improvements:

- Game statistic
  - Number of hands played
  - Contracts made
  - Contracts defeated
  - Tricks won by each partnership
  - Total score by team
- Multiple rounds
- AI or computer-controlled players

Functionally, we could have implemented more rules of the official game, such as doubling and redoubling during the bidding phase and some scoring features such as vulnerability.

Additionally, we could have added a graphical user interface to hide the hands of all players. Although our code is functionally correct in adherence to the rules, it lacks the practicality of hiding all players' hands (although the dummy's should always be visible).

If we were to expand our project, then retaining the separate game class would help prevent spaghetti code and reduce the clutter in our main file.

# Section F: Game Rules

Contract Bridge is a card game played by four players divided into two teams: North-South and East-West. The game uses a standard 52-card deck, with each player receiving 13 cards.

Each hand consists of two phases: bidding and playing.

During the bidding phase, players take turns either passing or making a bid. A bid specifies the number of tricks the team expects to win above six, as well as the suit that will be trump. A bid may also specify No Trump, meaning that no suit is designated as trump. The highest valid bid becomes the final contract. The team that makes the contract becomes the declaring team, while the opposing team attempts to prevent them from fulfilling it.

During the playing phase, the player who made the winning bid becomes the declarer and plays both their own cards and their partner's cards. The partner becomes the dummy, whose cards are placed face-up and are visible to all players.

Play proceeds clockwise around the table in the order North, East, South, West. The first player leads a card, and each following player must play a card of the same suit if they have one. If a player does not have a card of the led suit, they may play a card of another suit, including a trump card.

The highest-ranking card of the led suit wins the trick unless one or more trump cards are played. If a trump card is played, the highest-ranking trump card wins. The player who wins the trick becomes the leader of the next trick.

Play continues until all 52 cards have been played, resulting in 13 tricks. The number of tricks won by each team is then counted. The declaring team fulfills its contract if it wins at least the number of tricks specified by the contract (six plus the bid level). If it does not reach this number, the defending team has prevented the contract from being fulfilled.

# Section G: Team Responsibilities

All team members followed their responsibilities as outlined in the proposal.

## Jayden

Implemented the following classes: Card, Deck, Hand, Player

Responsible for submitting all files

Tested project

## Arianna

Implemented the following classes: Trick, scoreManager, Game (changed to main)

Tested project

Wrote final report

## Matthew

Implemented the following classes: Bid, biddingManager, contract

Tested Project

Sustenance for group members

## Section H: Appendix

## 1. Common Class (GR I, II, III)

### Enumerated Classes

1. Rank: ace is highest by convention in contract bridge, and no need for `#ifdef` statement
2. Suit: 0 = CLUBS, 1 = DIAMONDS, 2 = HEARTS, 3 = SPADES (suggested mapping based on Lab 1)
3. Position: North, East, South, West (clockwise)
4. Team: NorthSouth, EastWest

### Helper Functions

1. `suitoString`: displays suits from enum class as strings
2. `ranktoString`: displays ranks from enum class as strings
3. `nextPosition`: determines who the next player is, in clockwise order
4. `teamOf`: keeps track of teams

## 2. Card (GR I) Member functions:

- Setters: not necessary. Card value cannot change.
- Getters: `getrank`, `getsuit`
  - Should return rank and suit of *card*
  - No parameters. No values should be changed. `const` specifier added as a promise between the function and program that zero changes should be made.
- Constructors:
  - We used initialization lists, so that the values are assigned during the creation of the object instead of creating a blank object and then overwriting. This is trivial for small classes but can save on cost and resources for larger ones.
- Destructors: No dynamic memory is allocated, but it is good practice to prevent memory leaks.
- `rankValue`: converts enum list into integers to be compared later in the program

## 3. Deck (GR I) Member Functions:

- Constructor, Destructor: create and destroy deck
- `createDeck`: loops through ranks and suits to create full unshuffled deck
- `Shuffle`: shuffles cards using random number generator function
- `dealCard`: checks if deck is empty, identifies the last card in the back of the vector array, removes it, and returns it. Operation returns a single card. Is the base function for multiple cards to be dealt to each player in main.
- `isEmpty`: determines when *cards* is empty
- `intsize`: size of *cards*: determines the amount of cards remaining in the deck

## 4. Hand (GR III, VI, VIII) Member Functions

- **Constructors:** Create and destroy the hand. No manual memory management is required because the `vector` automatically manages its own memory.
- **addCard():** Adds one `Card` object to the hand using `push_back()`. Used when cards are dealt to a player.
- **removeCard():** Removes one card from the hand using its index.
  - First checks whether the index is valid.
  - If the index is invalid, throws an `out_of_range` exception.
  - If valid, removes the card from the vector.
  - Used when a player plays a card.
- **getCard():** Returns the card at a specified index without removing it.
  - Checks that the index is valid.
  - Throws an `out_of_range` exception if the index does not exist.
  - `const` because retrieving a card does not modify the hand.
- **hasSuit():** Checks whether the hand contains at least one card of a particular suit.
  - Loops through every card in the hand.
  - Returns `true` as soon as a card with the requested suit is found.
  - Returns `false` if no card has that suit.
  - Necessary for enforcing the Bridge rule that a player must follow the lead suit if possible.
- **isValidPlay():** Determines whether a selected card can legally be played according to the rules of Bridge.
  - **Case 1:** Checks whether the selected card index exists. Returns `false` if the index is invalid.
  - **Case 2:** Checks whether a suit has been led. If `leadSuit` is `NoTrump`, this is the first card of the trick, so any card can be played.
  - **Case 3:** Checks whether the selected card follows the lead suit. If it does, the play is valid.
  - **Case 4:** Checks whether the player has any cards of the lead suit. If they do not, they are allowed to play any suit.
  - If the player **has** the lead suit but attempts to play another suit, returns `false`.
- **sortHand():** Sorts the cards in the hand to keep them organized.
  - First sorts cards by suit.
  - If two cards have the same suit, sorts them by rank in ascending order.
  - Uses `rankValue()` because the rank enum needs to be converted into a numerical value for comparison.
- **displayHand():** Prints the player's hand to the terminal.
  - If the hand is empty, prints `"Hand is empty."`
  - Otherwise, prints each card's index, rank, and suit.
  - The index allows the player to select a specific card when making a play.
- **size():** Returns the number of cards currently in the hand.
  - Uses `cards.size()` to determine the number of stored `Card` objects.
  - Casts the result to an `int`.
- **isEmpty():** Determines whether the hand contains zero cards.
  - Returns `true` if the vector is empty.
  - Returns `false` if the hand contains at least one card.

## 5. Player (GR II, III, VII, VIII) Member Functions

- **Constructors:** Create a player and initialize their name, position, team, and hand.
  - Default constructor creates a player with an empty name, North position, and North-South team.
  - Parameterized constructor allows the name, position, and team to be specified.
  - Position constructor automatically determines the player's team using `teamOf()` and assigns the position as the player's name.
  - This allows players to be easily created based only on their Bridge position.
- **Destructor:** Destroys the player. No manual memory management is required because the player's `Hand` automatically manages its own `vector` of cards.
- **getName():** Returns the player's name.
  - `const` because retrieving the name does not modify the player.
- **getPosition():** Returns the player's Bridge position (`North`, `East`, `South`, or `West`).
  - `const` because retrieving the position does not modify the player.
- **getTeam():** Returns the player's team (`NorthSouth` or `EastWest`).
  - `const` because retrieving the team does not modify the player.
- **getHand():** Returns the player's current hand of cards.
  - Used when another part of the program needs to access the player's cards.
  - `const` because this function does not modify the player.
- **receiveCard():** Gives the player a card by passing the `Card` to their `Hand`'s `addCard()` function.
  - Used during the dealing phase of the game.
- **playCard():** Removes and returns a card from the player's hand.
  - Takes the index of the card the player wants to play.
  - First retrieves the card using `getCard()`.
  - Then removes that card using `removeCard()`.
  - Returns the played `Card` so it can be added to the current trick.
- **displayHand():** Displays the player's hand by calling the `Hand`'s `displayHand()` function.
  - Used to show the player which cards they currently hold.
- **hasSuit():** Checks whether the player has at least one card of a specified suit.
  - Calls the corresponding `hasSuit()` function from the player's `Hand`.
  - Used by the Bridge game to determine whether the player is required to follow the lead suit.

## 6. Bid Member Functions(GR IV, V)

- **Constructors:** Create a **Bid** object.
  - Default constructor creates a **Pass** bid with level **0** and **NoTrump** strain.
  - Normal bid constructor creates a normal bid using a specified level and strain.
  - Third constructor allows the bid type to also be specified.
  - Initialization lists are used to initialize the member variables when the object is created.
- **makePass():** Creates and returns a **Bid** representing a pass.
  - Used when a player chooses not to make a bid.
- **getType():** Returns the type of bid, such as **Pass** or **Normal**.
  - **const** because retrieving the value does not modify the bid.
- **getLevel():** Returns the numerical level of the bid.
  - In Bridge, normal bids have levels from **1 to 7**.
- **getStrain():** Returns the suit or **NoTrump** associated with the bid.
  - The possible strains are Clubs, Diamonds, Hearts, Spades, and No Trump.
- **isPass():** Checks whether the bid is a pass.
  - Returns **true** if the bid's type is **Pass**.
- **isNormal():** Checks whether the bid is a normal contract bid.
  - Returns **true** if the bid's type is **Normal**.
- **isHigherThan():** Compares two bids to determine which one is higher.
  - A normal bid is higher than a pass.
  - Higher **levels** outrank lower levels.
  - If two bids have the same level, their strains are compared.
  - The strain order is:  
**Clubs < Diamonds < Hearts < Spades < No Trump.**
  - Used to determine which bid is currently the highest valid bid during bidding.
- **isValid():** Checks whether the bid is structurally valid.
  - Passes are always considered valid.
  - Normal bids must have a level from **1 to 7**.
  - The strain must be one of Clubs, Diamonds, Hearts, Spades, or No Trump.
  - **Important:** this checks whether the bid itself has valid values; it does **not** determine whether it is higher than the previous bid. **isHigherThan()** handles that comparison.
- **toString():** Converts the bid into readable text for display.
  - A pass becomes **"Pass"**.
  - A normal bid becomes something such as **"3 Hearts"** or **"4 No Trump"**.
- **operator==():** Determines whether two bids are equal.
  - First compares their bid types.
  - For non-normal bids such as **Pass**, two bids of the same type are considered equal.
  - For normal bids, both the level and strain must match.

## 7. Bidding Manager Member Functions (GR II, IV, V, VI)

- **Constructor:** Initializes the bidding manager.
  - Sets the highest bid to **Pass**.
  - Sets the highest bidder to **nullptr** because nobody has bid yet.
  - Sets the consecutive pass counter to **0**.
- **setPlayers():** Gives the bidding manager access to the four players.
  - Stores pointers to the players in a vector.
  - Used later to identify a player's partner.
- **reset():** Resets the bidding manager for a new hand.
  - Clears the highest bid.
  - Removes the highest bidder.
  - Clears the previous bid history.
  - Resets the consecutive-pass counter.
  - Keeps the existing player list because the players themselves have not changed.
- **submitBid():** Attempts to submit a player's bid.
  - First checks whether the bid itself is structurally valid using **isValid()**.
  - If the bid is a **Pass**, records it in the bid history and increases the consecutive-pass counter.
  - If it is a **normal bid**, checks whether it is higher than the current highest bid.
  - Rejects a normal bid that does not beat the current highest bid.
  - If accepted, records the bid, updates the highest bid and highest bidder, and resets the consecutive-pass counter.
  - This is the main function that **enforces the rules of the bidding process**.
- **isBiddingComplete():** Determines whether the bidding phase has ended.
  - If nobody has made a bid, all four players must pass for bidding to end.
  - If a bid has been made, three consecutive passes end the bidding.
  - This determines when the program should stop accepting bids and move to the contract/play phase.
- **getHighestBid():** Returns the highest valid bid currently on the table.
- **getHighestBidder():** Returns a pointer to the player who made the highest bid.
- **getBidHistory():** Returns the complete sequence of bids made during the bidding phase.
  - Stores both the player who made each bid and the bid itself.
  - Used to determine the declarer after bidding ends.
- **findPartner():** Finds a player's Bridge partner.
  - Determines the position directly across from the given player.
  - Searches through the players to find the player in that position.
  - For example, North's partner is South, and East's partner is West.
- **getContract():** Creates the final contract after bidding is complete.
  - If everyone passed, returns an empty/default **Contract**, meaning there is no contract.
  - Otherwise, obtains the final bid's level and strain.
  - Determines the declaring team from the highest bidder.
  - Determines the **declarer**.
  - Finds the declarer's partner to become the **dummy**.
  - Returns a **Contract** containing the level, strain, declaring team, declarer, and dummy.
- **determineDeclarer():** Determines which player becomes the declarer.
  - Searches the bid history in chronological order.
  - Looks for the **first player on the winning team who bid the final winning strain**.
  - Returns that player as the declarer.
  - This follows the Bridge rule that the declarer is the first player from the winning partnership to bid the final strain.

## 8. Contract Member Functions (GR VI)

- **Constructors:** Create a contract.
  - Default constructor creates an empty/no-contract state with level 0, NoTrump, and no declarer or dummy.
  - Parameterized constructor creates the final contract using the winning bid's level and suit, declaring team, declarer, and dummy.
  - Calls `calculateRequiredTricks()` to determine how many tricks the declaring team must win.
- **getLevel():** Returns the contract level.
  - For example, a 3 Hearts contract has a level of 3.
- **getTrumpSuit():** Returns the contract's trump suit/strain.
  - Can be Clubs, Diamonds, Hearts, Spades, or No Trump.
- **getDeclaringTeam():** Returns the partnership that won the contract.
- **getDeclarer():** Returns a pointer to the player who is the declarer.
- **getDummy():** Returns a pointer to the player who is the dummy.
- **getRequiredTricks():** Returns the minimum number of tricks the declaring team must win to fulfill the contract.
- **isNoTrump():** Checks whether the contract is a No Trump contract.
  - Returns `true` if the trump suit is `Suit::NoTrump`.
- **calculateRequiredTricks():** Calculates the number of tricks required to make the contract.
  - Bridge contracts start with **6 tricks**.
  - The contract level determines how many additional tricks are required.
  - Formula:  
**Required tricks = 6 + contract level**
  - For example:
    - 1 Hearts → 7 tricks required
    - 3 Spades → 9 tricks required
    - 6 No Trump → 12 tricks required
- **display():** Displays the contract and its requirements to the terminal.
  - Displays the level and trump suit.
  - Displays the declarer's name if a declarer exists.
  - Displays the number of tricks required.

## 9. Trick Member Functions(GR VI, VIII, VIII, IX, X)

- **Constructors:** Create an empty trick.
  - Default constructor creates a trick with no lead suit and no trump suit.
  - Parameterized constructor creates a trick with the specified trump suit.
  - `leadSet_` starts as `false` because no card has been played yet.
- **addPlay():** Adds a player's card to the current trick.
  - If this is the first card played, its suit becomes the **lead suit**.
  - Stores the player and card together in `plays_`.
  - Used each time a player plays a card during the trick.
  - Supports **GR #7**.
- **followsSuit():** Checks whether a card follows the lead suit.
  - If no card has been played yet, there is no lead suit, so any card is valid.
  - Otherwise, checks whether the card's suit matches `leadSuit_`.
  - **Note:** This function only checks whether the card matches the lead suit; the more complete legal-play check is handled by `Hand::isValidPlay()`.
  - Supports **GR #8**.
- **determineWinner():** Determines which player wins the current trick.
  - Returns `nullptr` if no cards have been played.
  - Starts by assuming the first player played the winning card.
  - Compares each subsequent card against the current winning card.
  - **Trump cards:** A trump card beats a non-trump card.
  - If both cards are trump, the higher-ranked trump card wins.
  - **Non-trump cards:** Cards following the lead suit beat cards that do not follow the lead suit.
  - If both cards follow the lead suit, the higher-ranked card wins.
  - Updates the winning player and card whenever a stronger card is found.
  - Returns the player who won the trick.
  - **Directly implements GR #9 — determination of the winner of each trick.**
- **clear():** Resets the trick so it can be reused for another trick.
  - Removes all previously played cards.
  - Resets the lead suit to `NoTrump`.
  - Sets `leadSet_` back to `false`.
- **getLeadSuit():** Returns the suit that was led at the beginning of the trick.
- **getTrumpSuit():** Returns the trump suit for the contract.
- **cardsPlayed():** Returns the number of cards currently played in the trick.
  - Used to determine whether all four players have played.
- **getPlays():** Returns the complete collection of players and cards played in the trick.
  - Allows other parts of the game to inspect the trick without modifying it.
  -

## 10. Main (I-XII)

Note: the main function is not a class. It is functionally the same as what Game class would have been though.

### Helper Functions:

- **clearInvalidInput():** Clears invalid input from `std::cin` so the program can continue accepting user input after an invalid entry.
- **waitForPlayer():** Prompts the players to pass the computer to the next player and press Enter when ready.
  - Allows four people to play privately on one computer without displaying another player's hand.
  - Supports the four-player game structure.
- **hidePreviousScreen():** Prints blank lines to move previous information off the visible terminal screen.
  - Used to reduce the chance that the next player can see the previous player's information.
- **positionToString():** Converts a `Position` enum into readable text such as "North" or "East".
  - Used for displaying player positions.
- **teamToString():** Converts a `Team` enum into readable text such as "North/South" or "East/West".
- **cardToString():** Converts a `Card` into readable text such as "Ace of Spades".
- **getSuitFromChoice():** Converts the user's numerical suit selection into the corresponding `Suit` enum.
  - Used during the bidding phase.
- **displayHighestBid():** Displays the current highest bid and the player who made it.
  - Used to show players the current bidding state.

### Bidding Functions:

- **getBidFromPlayer():** Collects a player's bid through user input.
  - Displays the player's hand and current highest bid.
  - Allows the player to either make a normal bid or pass.
  - Validates the entered level and suit.
  - Returns the resulting `Bid` object.
  - Works together with `Bid` and `BiddingManager` to implement **GR #4**.
- **runBidding():** Runs the complete bidding phase.
  - Determines the dealer and starting bidder.
  - Cycles through the four players in order.
  - Submits each player's bid to `BiddingManager`.
  - Rejects illegal bids and allows the player to try again.
  - Continues until bidding is complete.

- Creates and displays the final **Contract**.
- Implements the overall bidding process for **GR #4 and #5**.
- Works with **BiddingManager** to identify the declarer and dummy for **GR #6**.

### Trick-Playing Functions:

- **getLegalCardChoice():** Gets a card selection from the current player.
  - Displays the player's hand.
  - Shows the lead suit.
  - Checks that the selected index exists.
  - Calls **Hand::isValidPlay()** to ensure the player follows the rules of Bridge.
  - Rejects illegal plays and asks the player to choose again.
  - Implements the user-input portion of **GR #8**.
- **displayCardsInTrick():** Displays all cards that have been played in the current trick.
  - Shows which player played each card.
- **playOneTrick():** Runs one complete trick.
  - Creates a **Trick** object.
  - Determines the current player and playing order.
  - Gives each of the four players a turn.
  - Gets a legal card choice.
  - Removes the card from the player's hand using **playCard()**.
  - Adds the card to the **Trick**.
  - Determines the winner using **Trick::determineWinner()**.
  - Records the winning partnership using **ScoreManager**.
  - Returns the winning player's position, making them the leader of the next trick.
  - Directly contributes to **GR #7, #9, and #10**.
- **playContract():** Runs the entire 13-trick playing phase.
  - Determines the opening leader.
  - Uses the contract's trump suit.
  - Calls **playOneTrick()** exactly 13 times.
  - The winner of each trick becomes the leader of the next trick.
  - Implements the complete 13-trick sequence required by **GR #7**.

### Scoring and Result Functions:

- **displayDealResult():** Displays the results after the hand is complete.
  - Shows the number of tricks won by each partnership.
  - Determines whether the declaring team made or defeated the contract.
  - Displays the required number of tricks and actual tricks won.
  - Calls **ScoreManager** to award points.
  - Displays the updated match score.
  - Implements **GR #10, #11, and #12**.

### Dealing Function:

- **dealCards():** Deals cards to the four players.
  - Creates a new 52-card deck.
  - Randomly shuffles the deck.
  - Deals 13 cards to each player by cycling through the four players 13 times.
  - Implements **GR #1 and #3**.

#### **main():**

- **main():** Controls the overall game flow.
  - Collects the four players' names.
  - Creates the four players and assigns them to their positions and partnerships.
  - Creates and shuffles the deck.
  - Deals 13 cards to each player.
  - Runs the bidding phase.
  - Creates the final contract.
  - Runs the 13-trick play phase.
  - Counts tricks and determines whether the contract was made.
  - Displays the result and score.
  - Allows players to play additional deals.
  - Rotates the dealer between deals.
  - Displays the final match score and winning partnership.